

Course plan v4 (draft for discussion) - one module per box of the kit (21 September 2026)

A proposal to replace the v3 layout (8 modules / 40 lessons, six-part page format inside every lesson) with a layout where **a module is one subsystem of code/** and **the six parts become the lessons of the module**. Nothing here is authored yet. `course-manifest.json` still carries v3.

1. The three rules

1. **A module is a box on DocuMind's diagram.** Every module maps to a folder, a service or a lane of `code/` and to the make targets that run it. If the kit does not contain it, the course does not teach it.
2. **The lessons of a module follow one rhythm - the four beats.** v3 put six parts on every page; v4 spreads them over the module so each lesson has one job:

Beat	Lesson type	v3 parts it absorbs	What the page holds
Why	Architecture and the decision	1 Architecture + database design, 3 Tech stack	The problem, the alternatives rejected, the Firestore / Spanner / Cloud SQL design, how an update is handled, what it bills
Where	Component diagram and flow	2 Component diagram, 4 Flow diagram	The box on the master diagram, what calls it, what it calls, what it reads and writes, the log lines it leaves
How	Code summary and code explanation	5 Code summary, 6 Code explanation	File by file: the load-bearing functions, the contracts, the gate that pins them
Do	Practice lab	(the practice lab)	Run it with a make target or notebook, break it on purpose, prove it with the smoke or the gate, price it in rupees

A small subsystem merges Why + Where into one lesson (3 lessons). A large one splits How in two, or adds a fifth **Operate / Compare** beat (5 lessons). Never 2, never 6.

1. **Every module ends on a gate the learner can run.** `make smoke-*`, `make eval`, `make eval-live`, `make dryrun`: the module is done when its gate is green. The rupee cost of the module is stated on its Why page.

Suggested time per beat: Why 60 min, Where 60 min, How 90 min, Do 120 min. A four-beat module is about 5.5 hours - one weekend (persona: ~6 h a week, evenings and weekends, IST).

2. The map - 15 modules, 64 lessons

#	Module	Box of the kit	Beats	Lessons	Gate
1	Set up and the Rs 0 lane	<code>evals/</code> , <code>shared/profile.py</code> , <code>shared/local_corpus.py</code> , <code>validate.py</code>	Why+Where, How, How, Do	4	<code>make dryrun</code> green; <code>make chat-local</code> answers
2	Infrastructure as code: Terraform, accounts, the data plane	<code>terraform/</code> , <code>commands/infrastructure.py</code>	Why+Where, How, How, Do	4	<code>make up</code> then <code>make smoke</code> ; <code>make down</code> leaves nothing billing

#	Module	Box of the kit	Beats	Lessons	Gate
3	Ingest: chunks, embeddings and the Firestore ledger	services/ingest/, shared/documind_corpus.py, shared/pii.py	Why, Where, How, How, Do	5	make smoke-reindex; the undo costs zero embeddings
4	Retrieval: vector, hybrid, the graph on Spanner, the managed stores	services/rag-api/retriever.py, hybrid.py, shared/documind_graph.py, services/ingest/graph.py, managed.py	Why+Where, How, How, How, Do	5	make ablate recorded; make smoke refuses a fallback answer
5	Generation: the cited answer, guarded and priced	services/rag-api/generator.py, context_budget.py, guard.py, cost.py, main.py, shared/documind_schemas.py	Why, Where, How, Do	4	make smoke; an injection blocked in English and Hinglish
6	Caches: the context cache and the answer cache	cache_manager.py, cache_admin.py, semantic_cache.py, evals/cache_threshold.py	Why+Where, How, Do	3	the second ask costs Rs 0
7	Evaluation: the gate, the judge, the ablation	evals/run_eval.py, judge.py, ablate.py, build_golden.py	Why+Where, How, How, Do	4	make eval-live green on a candidate
8	Identity, tenancy and the two screens (UI and admin)	shared/iap.py, shared/tenancy.py, shared/audit_log.py, services/frontend/, services/admin/	Why, Where, How, How, Do	5	isolation 1.00; the outsider refused
9	Agents inside the kit: four brains, one tool, memory on Cloud SQL	services/chat/, shared/documind_tools.py, terraform/cloudsql.tf	Why, Where, How, How, Do	5	make smoke-chat; a conversation survives a restart
10	MCP and A2A: agents outside the kit	services/mcp/, services/agent/	Why+Where, How, How, Do	4	make smoke-mcp and make smoke-agent
11	Multimodal: media is a document	ingest media path, evals/build_media.py, frontend/citations.py, rag- api/media.py, studio.py, voice.py	Why, Where, How, Do	4	make smoke-media; the mm- golden rows pass
12	Fine-tuning: Gemini SFT on Vertex AI, and your own small model	evals/make_trainset.py, evals/tune.py, evals/sft/, services/slm/make_modelfile.py	Why+Where, How, How, Do	4	the tuned candidate passes make eval-live; pairwise judged
13	The gateway and the GPU: LiteLLM, Ollama on Cloud Run, vLLM, GKE	services/litellm/, services/slm/, services/gemma-vllm/, terraform/gke.tf, gke/	Why, Where, How, How, Do	5	make smoke-gateway, make smoke-slm; make slm-off shows zero
14	The bill: routing, breakers, rupees per tenant, the night switch	router.py, breakers.py, budget.py, evals/usage_rows.py, services/admin/, sink.tf, alerts.tf, budget.tf, off.tf	Why+Where, How, How, Do	4	tenant_daily returns rupees; the off job floors the GPUs
15	Ship without keys, roll back in two minutes	cloudbuild.yaml, wif.tf, clouddeploy.tf, .github/workflows/, the release targets	Why+Where, How, Do, Operate	4	rollback under two minutes; make smoke-all green

Totals: 15 modules, 64 lessons, about 90 hours at the beat times above. Modules 2, 3, 13, 14 and 15 are make-driven (Cloud Shell or a workstation); the rest run from Colab against the deployed lane, with Module 1's Rs 0 lane as the fallback for every agent and MCP lesson.

Two dials, if the total is wrong:

- **Drop the optional lanes** the kit itself marks optional - vLLM + GKE (13.4 shrinks, 13.5 loses two steps), the A2A peer (10.3), the GGUF small model (12.3) - and the count is 15 modules / 60 lessons.
- **Merge Why + Where everywhere** (one architecture-and-diagram lesson per module) and the count is 15 modules / 56 lessons, every module at 3 or 4.

Order of the modules is the order a document travels (1-6), then proof (7), then the people and agents who ask (8-10), then media (11), then the model itself (12-13), then running it (14-15). Module 7 sits before the surfaces on purpose: the gate exists before anything is built on top of the pipeline, and every later module's Do lesson reuses it.

3. The modules and their lessons

Beat tags: **W** Why (architecture and the decision) · **D** Where (component diagram and flow) · **H** How (code) · **P** Do (practice) · **O** Operate / Compare. "Code it opens" names the files the page walks through and the make targets the lab runs.

Module 1 - Set up and the Rs 0 lane

By the end: a billable project, the kit on your machine, DocuMind answering on the laptop for Rs 0, and the ten offline checks green. Bills: nothing until 2.4.

#	Beat	Lesson	What the learner does	Code it opens
1.1	W+D	DocuMind in one diagram	The master diagram every later module zooms into: seven Cloud Run services, the stores, the two journeys (upload to index, question to answer). Then a throwaway project, billing, the APIs, ADC.	README.md, INDEX.md, Makefile (header), make apis, commands/infrastructure.py prepare, INFRASTRUCTURE.md
1.2	H	The code on your machine	Clone the learner repo; Python, Terraform, gcloud, Docker; the tour of services/, shared/, terraform/, evals/, smoke/, commands/; what the tool checks refuse.	commands/session-restart.sh, shared/requirements.txt, services/*/requirements.txt
1.3	H	The documents DocuMind reads	Three tenants, thirteen real Acts, invented PANs, the 65 golden questions and their five shapes; the one loader and chunker every lane shares.	evals/README.md, evals/fetch_real.py, build_corpus.py, build_golden.py, golden.jsonl, required.json, shared/documind_corpus.py
1.4	P	Run it for Rs 0, then the health check	Ollama + Chroma; ask "What is the notice period?"; then the ten offline checks - break one on purpose.	make chat-local, shared/profile.py, shared/local_corpus.py, services/chat/requirements-local.txt, make dryrun, validate.py, evals/run_eval.py (offline), .github/workflows/documind-dryrun.yml

Module 2 - Infrastructure as code: Terraform, accounts, the data plane

By the end: the whole of DocuMind stood up on a throwaway project from a reviewed plan, smoked, and torn down. Bills by the hour while up: the Vector Search endpoint, two Cloud SQL instances, Spanner, the GKE node.

#	Beat	Lesson	What the learner does	Code it opens
2.1	W+D	Why Terraform, and what bills	The resource map as a diagram (14 accounts, 5 buckets, Firestore and its indexes, Vector Search, two Cloud SQL, Spanner, BigQuery, registry, network); where state lives; why the plan is saved, reviewed, and refuses deletes.	terraform/backend.tf, variables.tf, commands/infrastructure.py, INFRASTRUCTURE.md
2.2	H	Who may do what	One account per service, the smallest roles, per-account actAs (why project-scope actAs was the defect), secrets never in env vars, the org policy that forbids keys.	terraform/sa.tf, secrets.tf, org_policy.tf, make secrets
2.3	H	The data plane, file by file	Every store the later modules write: Firestore + 5 composite indexes + 3 TTL fields, the Vector Search index (empty on purpose) and endpoint, the buckets, the Doc AI processor, Cloud SQL x2, Spanner, the BigQuery datasets, Pub/Sub, registry, VPC connector.	firestore.tf, firestore_indexes.tf, vector.tf, storage.tf, docai.tf, cloudsql.tf, spanner.tf, eventarc.tf, dataplex.tf, registry.tf, network.tf

#	Beat	Lesson	What the learner does	Code it opens
2.4	P	Plan, review, apply, tear down	make <code>plan</code> , read the plan, make up (images, seven services, IAP, roster, wait-vectors), make <code>smoke</code> , then make <code>down</code> and read what only project deletion removes. IAP, the roster and Cloud Build are used here and explained in Modules 8 and 15.	make <code>plan</code> , make <code>up</code> , <code>cloudbuild.yaml</code> , <code>commands/lesson-12.*.sh</code> , make <code>roster</code> , make <code>smoke</code> , make <code>down</code>

Module 3 - Ingest: chunks, embeddings and the Firestore ledger

By the end: a document goes in once, a re-issue pays only for what changed, an undo costs nothing, and every step leaves a log line. Bills: Document AI per page, DLP per byte, embeddings per token; a full corpus load is a few hundred rupees.

#	Beat	Lesson	What the learner does	Code it opens
3.1	W	Idempotent by design: the eight principles	Three identities (source, version, chunk); change by hash and generation, never by time; pay for what changed; stage then swap; retire, never delete; one chunker everywhere. The Firestore design: <code>chunks</code> , <code>documents</code> , <code>sources</code> , <code>ledger</code> , <code>ingest_batch</code> - and how an update is handled (281 reused, 2 embedded, 283 retired).	<code>INDEXING.md</code> (sections 1, 4, 8), <code>services/ingest/contracts.py</code> (docstrings)
3.2	D	From the bucket to the row	The pipeline diagram: <code>object.finalized</code> , <code>topic</code> , push subscription with OIDC, five attempts then the DLQ; the worker; <code>pypdf</code> count; Document AI in 15-page slices by residency; DLP; chunk by section or by window; embed regionally; Firestore staged; the swap; Vector Search dense+sparse with restricts; the BigQuery mirror; the managed mirror. The batch lane over 250 pages, the 23:30 reconcile, the reader's <code>current</code> pre-filter. The event names as the trail.	<code>terraform/eventarc.tf</code> , <code>batch.tf</code> , <code>reconcile.tf</code> , <code>INDEXING.md</code> (section 2), <code>evals/upload.sh</code>
3.3	H	From bytes to rows	The contracts (<code>IngestMessage</code> , <code>DocumentContract</code> , the tenant-scoped <code>chunk_id</code> , <code>chunk_hash</code> , <code>is_stale</code> , <code>effective_from_of</code>); the parser; the one PII list and its quota; the chunker; embedding in batches with the carry-over by hash; the datapoint shape; the Firestore and BigQuery mirrors.	<code>contracts.py</code> , <code>parser.py</code> , <code>shared/pii.py</code> , <code>shared/documind_corpus.py</code> , <code>indexer.py</code>
3.4	H	The ledger	The claim as a transaction, <code>finish</code> and <code>release</code> , the generation guard, <code>swap_versions</code> , <code>retire_previous</code> , the verified undo (<code>reactivate</code>), the tombstone (<code>withdrawn</code>), <code>record_source</code> , the corpus fingerprint; the worker's <code>push</code> and <code>index_document</code> ; the batch consumer; the reconcile plan and its <code>drift</code> number.	<code>idempotency.py</code> , <code>main.py</code> , <code>batch.py</code> , <code>reconcile.py</code> , <code>operators/remirror_missing.py</code>
3.5	P	Ingest, re-issue, undo, poison	make <code>ingest-one</code> ; make <code>reindex</code> and read <code>reused</code> / <code>embedded</code> / <code>retired</code> on the <code>ingest_ok</code> line; make <code>retire</code> and make <code>restore</code> ; make <code>poison</code> then make <code>dlq</code> ; make <code>queued</code> and make <code>batch</code> ; make <code>reconcile</code> ; make <code>smoke-reindex</code> .	make <code>ingest-one</code> / <code>reindex</code> / <code>retire</code> / <code>restore</code> / <code>poison</code> / <code>dlq</code> / <code>queued</code> / <code>batch</code> / <code>reconcile</code> / <code>sources</code> / <code>smoke-reindex</code> , <code>evals/demo/</code>

Module 4 - Retrieval: vector, hybrid, the graph on Spanner, the managed stores

By the end: the learner can name the four places a question can be answered from, switch a tenant between them with a setting, and show with numbers which one is better. Bills: the Vector Search endpoint by the hour, Spanner by the hour, RAG Engine and Vertex AI Search storage while a corpus exists.

#	Beat	Lesson	What the learner does	Code it opens
4.1	W+D	Four places to search, and who chooses	The ANN tier and the Firestore rung beneath it, RAG Engine, Vertex AI Search; the tenant pin and the fallback; why hybrid (dense + sparse, RRF); why rerank; current-only and newest-per-source; data residency (tenant_settings.data_region); when a knowledge graph pays. The query-path diagram: embed, choose backend, search, prefer current, graph candidates, rerank, pool; stages.retrieval_backend and found_by on every answer.	INDEXING.md (section 3), services/rag-api/config.py, shared/tenancy.py (backend_for, policy_for), evals/ablate.py (docstring)
4.2	H	Dense, hybrid, rerank	embed_query with the one declared model; the Firestore fallback; newest_per_source and prefer_current; retrieve; the reranker and rerank_fell_back; RRF in-process and server-side; the shared sparse encoder.	retriever.py, hybrid.py, shared/sparse_encoder.py, commands/verify-vector-index.py, commands/check-firestore-fallback.py
4.3	H	The knowledge graph, on Firestore and on Spanner	Entities and relations stated in the passage, never inferred; resolving surface forms by embedding at 0.92; nodes and edges in graph_nodes / graph_edges, or in Spanner's GraphNode / GraphEdge (interleaved, with an embedding column) seeded by meaning with COSINE_DISTANCE; the walk in front of the dense pool.	services/ingest/graph.py, shared/documind_graph.py, terraform/spanner.tf, retriever.py (graph_candidates, embed_for_graph), commands/tests/test_spanner_graph.py
4.4	H	The managed mirrors	The rule that the ledger is the source of truth and a store holds current versions only; the RAG Engine corpus and the Vertex AI Search data store as mirrors written after the swap and after the undo; what a forbidden store looks like in the log; status against the ledger.	services/ingest/managed.py, terraform/managed.tf, make rag-engine-enable, make rag-corpus, make managed-status, course-bibles/managed-retrieval-plan-2026-09-13.md
4.5	P	Measure it, switch it, break it	make ablate (four arms against the golden set: recall@k, MRR, p95); make tenant-backend TENANT=acme RETRIEVAL_BACKEND=firestore then rag_engine; make graph GRAPH_BACKEND=spanner and make candidate RETRIEVAL_GRAPH=auto; the chaos drill (undeploy the index, still get an answer, found_by: firestore); make backfill-vectors.	make ablate / tenant-backend / tenant-policy / graph / candidate / vector-status / wait-vectors / backfill-vectors, make smoke

Module 5 - Generation: the cited answer, guarded and priced

By the end: an answer that cites, refuses when it cannot, is screened on both sides, streams, and carries its own rupee cost. Bills: Gemini per token; Model Armor per screened request.

#	Beat	Lesson	What the learner does	Code it opens
5.1	W	One contract, one budget, one price	The answer contract (Citation, RAGAnswer, the model's DraftCitation resolved to a real chunk); most-relevant-first packing into a token budget; JSON out of the model; refusing on an empty pool; Model Armor on both sides; the two clients (generation on global, embeddings regional); a tuned endpoint as a setting; INR at 85 on every answer.	shared/documind_schemas.py, context_budget.py (docstring), cost.py, terraform/model_armor.tf
5.2	D	/v1/query and /v1/stream	The request path as a diagram with its stages (retrieve, rerank, generate), the SSE citation event, the usage row (one shape on every answer), the guard's two positions, /version, /ready, /v1/sources; the UI as a thin client that never holds a model.	main.py (routes, stage, usage_row, empty_pool_answer), schemas.py, telemetry.py, services/frontend/chat.py (stream_answer)
5.3	H	The generator	pack_chunks and source_header; generate and generate_stream; _client_for and _endpoint_location; the gateway reply; the dated rule that appears only when a packed source carries a date; check_prompt and check_response; price.	context_budget.py, generator.py, guard.py, cost.py, rag-api/auth.py

#	Beat	Lesson	What the learner does	Code it opens
5.4	P	Ask it, stream it, attack it, price it	<code>curl /v1/query</code> with an ID token; stream and watch the citation event; make candidate <code>ARMOR=on</code> and inject in English and Hinglish (a 400 <code>prompt_blocked</code>); an unanswerable question refused; make usage <code>HOURS=1</code> for the rupee line.	<code>make candidate</code> , <code>smoke/smoke.py</code> , <code>make usage</code> , <code>commands/lesson-12.2.sh (ARMOR)</code>

Module 6 - Caches: the context cache and the answer cache

By the end: the second identical question costs Rs 0, a reindex makes every earlier answer a miss, and the similarity threshold is a measured number, not a guess. Bills: cache storage by the hour until deleted.

#	Beat	Lesson	What the learner does	Code it opens
6.1	W+D	Two caches, one ledger	What an explicit Gemini cache buys (the 4,096-token floor, the 60-minute default, no maximum, storage billed by the hour); why it lives on <code>global</code> ; why the cache follows the corpus fingerprint (<code>cache_stale</code>); the semantic answer cache: a hit with its original citations at cost 0, the risk of a right-sounding wrong answer, the TTL by Firestore policy. The diagram: <code>tenant_caches/{tenant}</code> , <code>answer_cache</code> , <code>ledger/{tenant}</code> , the request path before retrieval.	<code>cache_manager.py</code> (docstring), <code>semantic_cache.py</code> (docstring), <code>INDEXING.md</code> (section 3, the cache paragraph)
6.2	H	The cache code	<code>TenantCacheManager</code> and <code>generate_config_kwargs</code> ; <code>pack_for</code> , <code>create / show / refresh / delete</code> ; <code>qhash</code> , <code>scope_of</code> , <code>lookup</code> , <code>store</code> ; the <code>SEMANTIC_CACHE</code> branch in the API; the threshold script and the paraphrase pairs.	<code>cache_manager.py</code> , <code>cache_admin.py</code> , <code>semantic_cache.py</code> , <code>main.py</code> (the cache branch), <code>evals/cache_threshold.py</code> , <code>evals/paraphrases.jsonl</code>
6.3	P	Pack it, hit it, stale it	make cache <code>TENANT=acme</code> then read <code>cached_tokens</code> on the next usage row; <code>CACHE_OP=show / refresh / delete</code> ; a reindex then <code>cache_stale</code> in the log; <code>cache_threshold.py --project</code> and choose the threshold from the curve; make candidate <code>SEMANTIC_CACHE=on</code> , ask twice, <code>model_backend=cache</code> .	<code>make cache</code> , <code>make reindex</code> , <code>make candidate</code> , <code>make usage</code>

Module 7 - Evaluation: the gate, the judge, the ablation

By the end: a golden set that can go red, a gate that blocks a merge, a judge that explains, and a habit of running both before every later module's Do lesson. Bills: the judge's Gemini calls; the live gate's answers.

#	Beat	Lesson	What the learner does	Code it opens
7.1	W+D	A gate is not a judge	The golden set's five shapes (<code>lookup</code> , <code>join</code> , <code>refusal</code> , <code>isolation</code> , <code>version</code>); the offline half in CI and the live half against a no-traffic candidate; nine thresholds and fifteen required rows; why isolation is 1.00; the two identities (<code>member</code> , <code>outsider</code>); a judge that scores groundedness and compares pairwise but never gates. The diagram: <code>pull request</code> , <code>dryrun</code> , <code>build</code> , <code>candidate</code> , <code>eval-live</code> , <code>judge</code> , <code>promote</code> .	<code>evals/README.md</code> , <code>golden.jsonl</code> , <code>required.json</code> , <code>manifest.json</code> , <code>run_eval.py</code> (docstring), <code>judge.py</code> (docstring)
7.2	H	The gate	<code>build_golden.py</code> ; <code>run_eval.py</code> offline (<code>anchors</code> , <code>coverage</code> , <code>falsifiability</code>) and live (the <code>outsider token</code> , <code>--source</code>); make candidate; the CI workflow that runs the offline half on every push.	<code>build_golden.py</code> , <code>run_eval.py</code> , <code>make eval</code> , <code>make eval-live</code> , <code>make candidate</code> , <code>.github/workflows/documind-dryrun.yml</code>
7.3	H	The judge and the ablation	Pointwise groundedness over the same answers, pairwise against the live revision, trajectories over the chat brains, one Experiments run per commit; the ablation harness with one knob per arm and no model in the loop.	<code>judge.py</code> , <code>ablate.py</code> , <code>make judge</code> , <code>make ablate</code>

#	Beat	Lesson	What the learner does	Code it opens
7.4	P	Make it go red	make eval offline; loosen a must_contain and watch the diff; make candidate then make eval-live API=; read the miss list; make judge API_B= pairwise; make ablate ABLATE_ARGS="--limit 5".	make eval / candidate / eval-live / judge / ablate, evals/demo_corpus_gate.py

Module 8 - Identity, tenancy and the two screens (UI and admin)

By the end: a person reaches DocuMind through IAP, a tenant-B question cannot return a tenant-A chunk, the outsider is refused everywhere, and the admin console shows who spent what. Bills: nothing new.

#	Beat	Lesson	What the learner does	Code it opens
8.1	W	The tenant is something you are	The roster in Firestore (tenants/{id}/members/{email}); one IAP verifier with two legs (the forwarded assertion, the caller's own ID token); a header is a claim, not a credential; the admin as its own service and account; one PII list for the worker and the dashboard; one audit event shape into a retention-locked bucket.	shared/iap.py (docstring), shared/tenancy.py (docstring), shared/audit_log.py, UNOWNED.md (the shared modules)
8.2	D	Seven accounts, three rosters	The surfaces diagram: person, IAP, the UI as ui-sa, the API verifying and checking the roster; the UI calling chat, MCP and the agent with ID tokens; the admin reading BigQuery, Firestore and the audit index; the outsider account; which account sits on which roster and why.	terraform/sa.tf (the roster grants), commands/lesson-12.1.sh, lesson-12.3.sh, lesson-12.4.sh, make roster
8.3	H	The identity code	accepted_audiences, verify, identity; is_member, tenant_for, add_member, policy_of, backend_for, set_policy; emit; the three auth.py files that now share one verifier; the gate that pins the seams.	shared/iap.py, shared/tenancy.py, shared/audit_log.py, rag-api/auth.py, admin/auth.py, frontend/auth.py, tools/check_auth_wiring.py
8.4	H	The Streamlit UI and the admin console	app.py and the four pages; chat.py as a thin streaming client with the brain radio; citations.py (text, figure, clip); documents.py (upload, the versions list from /v1/sources); the Studio and voice pages as clients of Module 11; the admin dashboard's five tabs (usage, tenants, audit, DLP, ingestion) and why the UI only links to it. Streamlit, not React - see section 6.	services/frontend/*, services/admin/admin_dashboard.py, admin/dlp.py, admin/audit.py, admin/app.py
8.5	P	Log in, be refused, be counted	make roster MEMBERS=; open the UI behind IAP; ask as acme, ask as zeta, prove isolation; smoke.py refuses the golden question without a token; the outsider gets 403 on every surface; make tenant-policy TENANT=globex DATA_REGION=in; the DLP tab shows the invoice's PAN finding; the audit tab shows the upload.	make roster, make tenant-policy, smoke/smoke.py, make eval-live (isolation rows)

Module 9 - Agents inside the kit: four brains, one tool, memory on Cloud SQL

By the end: the same question through a hand-written loop, LangChain, LangGraph and ADK, over one retrieve(), with a conversation that survives a restart and a cost line per brain. Bills: the chat Cloud SQL instance by the hour; Gemini per turn.

#	Beat	Lesson	What the learner does	Code it opens
9.1	W	Every difference is the harness	Why retrieval lives once (documind_tools.retrieve, the drift watched four times); a direct loop as the floor; LangChain's create_agent; a LangGraph StateGraph with a refuse node; ADK's LlmAgent and Runner; why memory needs a checkpoint, why Cloud SQL, why migrations are a one-off job; the tenant is not a request field.	brains.py (docstring), chat/agent.py (docstring), migrate.py, shared/documind_tools.py (docstring), tools/check_one_retrieval.py

#	Beat	Lesson	What the learner does	Code it opens
9.2	D	One <code>/v1/chat</code> , four brains, one memory	The diagram: the UI's brain radio, <code>/v1/chat</code> (IAP, roster, thread id), the brain, <code>tools.py</code> , <code>retrieve()</code> calling the API as chat-sa with the person's assertion forwarded; PostgresSaver on Cloud SQL; ADK's DatabaseSessionService on the same instance; the usage row naming the brain; the same service on the laptop with <code>CHECKPOINT_DSN=memory</code> .	<code>terraform/cloudsql.tf</code> , <code>commands/lesson-12.8.sh</code> , <code>shared/profile.py</code> , <code>services/chat/Dockerfile</code>
9.3	H	The tool, the direct loop, the LangChain brain	<code>retrieve</code> and <code>calculate_processing_cost</code> , the ID token minted per call; the chat adapter (<code>retrieve</code> bound to the tenant, <code>BLOCKED</code> , <code>TIMEOUTS</code>); <code>DirectBrain</code> ; <code>LangChainBrain</code> with the guard as middleware; <code>_summary</code> - the same three keys from every brain.	<code>shared/documind_tools.py</code> , <code>chat/tools.py</code> , <code>brains.py</code> (the first two brains)
9.4	H	LangGraph, ADK and the checkpointer	<code>LangGraphBrain</code> (agent, tools, refuse), <code>AdkBrain</code> (before_tool_callback, session service), <code>agent.py</code> (caller, thread_config, brain_for, lifespan, chat), <code>migrate.py</code> , the local requirements.	<code>brains.py</code> (the last two brains), <code>chat/agent.py</code> , <code>migrate.py</code> , <code>requirements-local.txt</code>
9.5	P	Four brains, one restart	<code>make smoke-chat</code> (health names four brains; <code>retrieve</code> among the tool calls); the "conversation survives a restart" check; the same service on the laptop with <code>make chat-local</code> , switching brains per request; <code>make judge</code> <code>CHAT_URL=</code> for the trajectories; the four cost lines side by side.	<code>make smoke-chat</code> , <code>smoke/smoke_chat.py</code> , <code>make chat-local</code> , <code>make judge</code> , <code>make usage</code>

Module 10 - MCP and A2A: agents outside the kit

By the end: `retrieve()` is a tool any agent can call over the network, it refuses the wrong tenant, and a peer agent that knows DocuMind only by one URL answers over A2A. Bills: nothing new.

#	Beat	Lesson	What the learner does	Code it opens
10.1	W+D	The UI is for people, MCP is for agents	Four tools, stateless streamable HTTP, the tenant from identity and roster and never from a tool argument, a server that owns no index; the A2A peer: one URL, its own account on one roster, an agent card and JSON-RPC; the identity chain as a diagram (A2A client, the peer, documind-mcp, the API); why the peer imports nothing from <code>shared/</code> .	<code>mcp/server.py</code> (docstring), <code>agent/agent.py</code> (docstring), <code>tools/check_one_retrieval.py</code>
10.2	H	The MCP server	<code>retrieve</code> , <code>list_documents</code> (filtered on the field, never the id prefix), <code>corpus_stats</code> , <code>calculate_processing_cost</code> , <code>health</code> ; the profile switch that runs it on the Rs 0 lane; the image and the deploy command.	<code>services/mcp/server.py</code> , <code>mcp/Dockerfile</code> , <code>commands/lesson-7.2.sh</code>
10.3	H	The A2A peer	<code>build_agent</code> : an <code>LlmAgent</code> over <code>McpToolset</code> with streamable-HTTP params, the header provider that mints a fresh token on every tool call, <code>to_a2a</code> and the agent card; an image that copies only its own directory; who may invoke it.	<code>services/agent/agent.py</code> , <code>agent/Dockerfile</code> , <code>commands/lesson-8.4.sh</code> , <code>terraform/sa.tf</code> (run.invoker)
10.4	P	Call it from outside	Run the server on the laptop against the Rs 0 lane; deploy; <code>make smoke-mcp</code> (<code>tools/list</code> , a member's <code>retrieve</code> , the refusal); connect Claude Desktop or an ADK client with an ID token; <code>make smoke-agent</code> (the card refused without a token, <code>message/send</code> completes); watch another tenant be refused by the server, not the peer.	<code>make smoke-mcp</code> , <code>smoke/smoke_mcp.py</code> , <code>make smoke-agent</code> , <code>smoke/smoke_agent.py</code>

Module 11 - Multimodal: media is a document

By the end: a chart, a scanned page and a video go through the same pipeline, answers cite the figure or the second of the clip, and the UI shows it. Bills: Gemini per described image or minute of video; DLP image inspection; Chirp per synthesised second; image generation per image.

#	Beat	Lesson	What the learner does	Code it opens
11.1	W	Verbalise before you embed	A vision-language model describes the picture and the caption is what retrieval sees; one contract for four modalities (figure, table, segment with kind, media_url, start, end); pixels DLP-scanned first, and why that scan runs in asia-southeast1; a citation is what the reader can open; the Studio spends on a tenant's behalf only after the roster; a 40 MB video is an ingest, not a stored file.	ingest main.py (MEDIA_TYPES, _describe_media docstring), frontend/citations.py (docstring), rag-api/media.py (docstring), shared/documind_schemas.py
11.2	D	From a figure to [Fig 3, p.12]	The diagram: make_media draws Figure 3 from its own table, renders the invoice and page 30 of the Bonus Act, synthesises the town hall MP4; the uploads bucket; the worker's media branch (Gemini from_uri, segments of at most 60 seconds); chunks rows with media fields; the same retrieve(); the figure Part shown to the model at generation; the UI rendering inline or starting the clip at its second. The Studio's /v1/media/generate and /v1/media/upload-url (a signed PUT), the TTS cache bucket, the media bucket.	evals/build_media.py (docstring), terraform/storage.tf (media, tts_cache), run_eval.py (media_kind_rate)
11.3	H	The media code	build_media.py (render, synthesise, ffmpeg); _describe_media and the image DLP branch of index_document; inspect_image; signed_url and render_with_citations; generate and upload_url with their usage and audit rows; the Studio and voice pages.	evals/build_media.py, services/ingest/main.py, shared/pii.py, frontend/citations.py, rag-api/media.py, frontend/studio.py, frontend/voice.py
11.4	P	Ask about the chart, then the clip	make_media MEDIA_ARGS=--video, make ingest-corpus; ask what revenue did in Q1 and get [Fig 3, p.12]; ask about the town hall and get [Clip 2, 03:20] with the video at that second; generate an image in the Studio and find its event=media usage row; make smoke-media (five legs); the mm- rows in make eval-live.	make_media, make ingest-corpus, make smoke-media, smoke/smoke_media.py, golden.jsonl (mm- rows)

Module 12 - Fine-tuning: Gemini SFT on Vertex AI, and your own small model

By the end: a model tuned on DocuMind's own documents, served as a setting on the API, tested by the gate and judged against the live one; and the files a small open model needs to be served in Module 13. Bills: the tuning job per training token; the tuned endpoint while deployed; nothing for the dataset.

#	Beat	Lesson	What the learner does	Code it opens
12.1	W+D	The dataset is a document, the model is a setting	One question and answer per real chunk, in the lane's citation grammar, PII-scanned, every golden question excluded, frozen with a manifest in the datasets bucket - never from the logs, never from the answer cache; GENERATOR_MODEL as a name on global or an endpoint path served from the location the path names (the us multi-region finding); tuning is regional and explicit; the small model's route: chat-format file, LoRA on a free T4, GGUF, a Modelfile generated from the tokenizer; the gate decides, the judge explains. The diagram: corpus, trainset, bucket, tuning job, endpoint, candidate, eval-live and judge; the GGUF handed to Module 13.	make_trainset.py (docstring), tune.py (docstring), evals/sft/documind_sft_v1.manifest.json, generator.py (_client_for, _endpoint_location)
12.2	H	The dataset and the managed job	load_chunks, sample, target, exclude_golden, redact, to_vertex, to_chat, the batch requests; config_for, launch, poll; the endpoint on a candidate.	evals/make_trainset.py, evals/tune.py, evals/sft/documind_sft_v1.vertex.jsonl, make_trainset, make_tune, make_candidate

#	Beat	Lesson	What the learner does	Code it opens
12.3	H	The small model's files	The chat-format file; why the Modelfile is generated from the tokenizer and never typed from a blog post (a template that disagrees with the weights answers fluently and never stops); the GGUF's one name and place; the training notebook on a free T4 (not in the kit today - see section 6).	evals/sft/documind_sft_v1.chat.js onl, services/slm/make_modelfile.py, services/slm/Modelfile
12.4	P	Tune it, gate it, judge it, price it	make trainset TENANT=acme ROWS=300 and read the manifest; make tune TUNE_BASE=gemini-3.1-flash-lite, poll; make candidate GENERATOR_MODEL=<endpoint path>; make eval-live API= and make judge API_B= pairwise; make usage to price the difference; generate the Modelfile from a checkpoint.	make trainset / tune / candidate / eval-live / judge / usage, make_modelfile.py

Module 13 - The gateway and the GPU: LiteLLM, Ollama on Cloud Run, vLLM, GKE

By the end: the same API answers from Gemini, from your own small model on an L4, or from vLLM, through one door; a PAN never leaves; and the GPU is off by 23:00. Bills: an L4 instance while warm (about Rs 86,904 a month if left on), the gateway's Cloud SQL by the hour, the GKE control plane and node while provisioned.

#	Beat	Lesson	What the learner does	Code it opens
13.1	W	The backend is a setting, the GPU is a bill	One door for every model and the ID token as the key (no master key); restricted text routed to the model that stays in India with no fallback; a token proxy because LiteLLM reads a key once and an ID token lives an hour; scale to zero and the 30-60 s cold start; a quota cap under every -max-instances; the Cloud Run versus GKE sum and the 74 percent duty-cycle break-even; why the tuned model's Modelfile must come from the tokenizer.	services/litellm/config.yaml (header), services/slm/gpu_quota.py (docstring), services/slm/Dockerfile (comments), gke/README.md
13.2	D	One door, three engines	The diagram: the API with MODEL_BACKEND=gateway, the gateway on Cloud Run behind IAM, the routes (general, reasoning, slm, sensitive, inference, gke), the token proxy, the SLM (Ollama on an L4), the vLLM service, the GKE pool on the lane's VPC; Postgres spend logs and tag budgets; the cost header into the usage row; a tenant pinned to a backend without a redeploy; the off job and the gpu_left_warm alarm.	terraform/gateway.tf, gke.tf, gke/vllm-deployment.yaml, off.tf, alerts.tf (the GPU policy), rag-api/generator.py (_GatewayReply)
13.3	H	The gateway	model_list, router_settings, the guardrail and the budgets; the sensitivity classifier and the router; the DLP audit; the token proxy and the ID-token cache per audience; the endpoint that starts both.	services/litellm/config.yaml, documind_router.py, documind_classifier.py, dlp_audit.py, token_proxy.py, gcp_id_token.py, endpoint.sh, Dockerfile
13.4	H	Two engines and a cluster	The SLM image (the tuned GGUF or the SLM_STOCK stand-in, created at build time), the quota tool, the comparison harness; the vLLM service (AsyncLLMEngine in lifespan, tiered API keys and rate limits, OpenAI-shaped schemas, SSE streaming, /classify and /extract, PII redaction in logs, the 13 GB image); the Autopilot manifest.	services/slm/*, services/gemma-vllm/*, terraform/gke.tf, gke/vllm-deployment.yaml
13.5	P	Through the door, onto the GPU, off by night	make deploy-gateway and make smoke-gateway (no token refused; a JSON completion; the cost header; a PAN re-routed; the SLM route); make deploy-slm and make smoke-slm (the cold start timed); make candidate MODEL_BACKEND=gateway GENERATOR_MODEL=documind-slm and make eval-live; make compare (the same questions on every backend, the four numbers); make gpu-cap; make slm-off. Optional: make build-vllm deploy-vllm, make gke-up and make gke-down.	make deploy-gateway / smoke-gateway / deploy-slm / smoke-slm / candidate / compare / gpu-cap / gpu-quota / slm-off / vllm-off / build-vllm / deploy-vllm / gke-up / gke-down

Module 14 - The bill: routing, breakers, rupees per tenant, the night switch

By the end: a request is routed by difficulty, the service degrades instead of failing at 80 and 100 percent of budget, every tenant's day is a rupee figure in BigQuery, an alarm pages when a GPU is left warm, and the lane is at zero by 23:00. Bills: BigQuery per query; nothing else new.

#	Beat	Lesson	What the learner does	Code it opens
14.1	W+D	Cost is a feature	Route by complexity; what the service does at 80 and 100 percent; the month's counter in Firestore; one usage row per answer (tenant, model, backend, brain, surface, INR at 85) into Cloud Logging first and BigQuery <code>tenant_daily</code> a day later; the admin console's tabs; six log metrics and six alert policies; the billing budget; the 23:00 job; <code>make down</code> . Two diagrams: the request path (router, breaker) and the reporting path (row, logging, sink, view, dashboard) with the switches.	<code>breakers.py</code> , <code>budget.py</code> , <code>evals/usage_rows.py</code> (docstring), <code>terraform/budget.tf</code> , <code>off.tf</code> , <code>alerts.tf</code> (the metric names)
14.2	H	On the request path	<code>Complexity</code> , <code>classify</code> , <code>route</code> ; <code>routing_mode</code> , <code>choose_model</code> ; <code>record</code> , <code>spend_pct</code> and the <code>SPEND_PCT</code> replay; <code>choose_model_for</code> and <code>usage_row</code> in the API; the Gemini quota override.	<code>router.py</code> , <code>breakers.py</code> , <code>budget.py</code> , <code>main.py</code> (<code>choose_model_for</code> , <code>usage_row</code>), <code>cost.py</code> , <code>terraform/quota.tf</code>
14.3	H	The reporting path and the switches	The usage tool's grouping and p95 per stage; the sink and the daily view; the feature job and the Dataplex quality scan over the chunk mirror; the admin dashboard's tabs; the metrics, policies and channels; the budget; the off job's account and schedule; the guarded spans.	<code>evals/usage_rows.py</code> , <code>terraform/sink.tf</code> , <code>sql/tenant_daily.sql</code> , <code>dataplex.tf</code> , <code>sql/chunk_metadata.sql</code> , <code>services/admin/admin_dashboard.py</code> , <code>alerts.tf</code> , <code>budget.tf</code> , <code>off.tf</code> , <code>rag-api/telemetry.py</code>
14.4	P	Route it, break it, count it, switch it off	<code>make candidate ROUTING=on</code> ; <code>SPEND_PCT=85</code> and watch the model tier change; <code>make usage HOURS=24</code> ; <code>make bq-views</code> and query <code>tenant_daily</code> ; <code>make features</code> ; open the admin console; trip an alert (a DLQ message, or a GPU left warm); <code>make off</code> and <code>make off-now</code> .	<code>make candidate / usage / bq-views / features / off / off-now / gpu-cap</code> , <code>make poison</code>

Module 15 - Ship without keys, roll back in two minutes

By the end: GitHub deploys to Google Cloud with no secret, a candidate is judged before it takes traffic, promotion is by name, rollback is to exactly the previous revision, and seven smokes run as one. Bills: Cloud Build minutes; a candidate revision while it exists.

#	Beat	Lesson	What the learner does	Code it opens
15.1	W+D	Keyless, gated, reversible	Workload Identity Federation pinned to the repository id and the branch; the org policy that removes the ability to create a key; Cloud Build from the <code>code/</code> context so a service can import <code>shared/</code> ; a no-traffic candidate; the eval gate before traffic; promote by revision name, never "latest"; rollback to the recorded previous revision; Cloud Deploy's canary as the other path. The diagram: push, the dryrun workflow, the CD workflow (WIF, Cloud Build, Artifact Registry, release-candidate, eval-gate, promote behind reviewers) and <code>release-clouddeploy</code> .	<code>terraform/wif.tf</code> , <code>org_policy.tf</code> , <code>clouddeploy.tf</code> , <code>.github/workflows/documind-cd.yml</code> (header)
15.2	H	The pipeline's files	<code>cloudbuild.yaml</code> ; <code>run-service.yaml</code> and its missing traffic stanza; the WIF pool and provider; the delivery pipeline and its two targets; the CD workflow's <code>path</code> input, its eval-gate job and the production environment; the dryrun workflow; the Makefile's <code>build</code> , <code>release-candidate</code> , <code>record-candidate</code> , <code>promote</code> , <code>rollback</code> , <code>smoke-all</code> and the two revision files.	<code>cloudbuild.yaml</code> , <code>run-service.yaml</code> , <code>terraform/wif.tf</code> , <code>clouddeploy.tf</code> , <code>.github/workflows/*.yaml</code> , <code>Makefile</code> (the release targets), <code>tools/check_release.py</code>
15.3	P	The release	<code>make build</code> ; <code>make release-candidate GIT_SHA=</code> ; <code>make eval-live API=<candidate URL></code> ; <code>make promote</code> ; <code>make smoke-all</code> ; then the same through <code>workflow_dispatch</code> .	<code>make build / release-candidate / eval-live / promote / smoke-all</code>

#	Beat	Lesson	What the learner does	Code it opens
15.4	O	The rollback and the runbook	Ship a candidate with a wrong setting, watch the gate refuse it; promote a bad one on purpose, make rollback, time it; then write the runbook: what pages, what the night job floors, what make down removes, what only project deletion removes.	make rollback, make down, smoke/preflight.sh, README.md (Tier B checklist)

4. Every store the kit uses, and the module that owns it

The databases were the user's first question, so here is the whole map. "Owns" is the module whose Why lesson carries the design and whose How lesson reads the code that writes it.

Store	What it holds	Owned by	Also read in
Firestore chunks	every chunk row: text, the dense vector, doc_key, current, chunk_hash, locator, the embedding stamp, staged / expire_at, media fields	3	4, 6, 11
Firestore documents	the per-version claim: processing, queued, indexed, superseded, failed; what a reindex cost	3	10 (list_documents)
Firestore sources	the ledger per object path: current doc_key, generation, sha256, counts, effective_from, withdrawn	3	8 (/v1/sources, the Documents page), 14
Firestore ledger/{tenant}	the corpus fingerprint both caches follow	3	6
Firestore ingest_batch	the batch lane's queue	3	-
Firestore tenants/{id}/members	the roster	8	9, 10, 11
Firestore tenant_settings	data_region, the retrieval backend pin, the model backend pin	4	8, 13
Firestore tenant_caches	the context cache record and its fingerprint	6	-
Firestore answer_cache	the semantic cache, reaped by a TTL policy	6	-
Firestore budget	the month's spend counter the breaker reads	14	-
Firestore graph_nodes, graph_edges, graph_extractions	the Firestore graph backend	4	-
Firestore dlp_findings, audit_index, api_keys	the admin console's fast indexes; the vLLM service's keys	8	13, 14
Vector Search index + endpoint	dense + sparse datapoints with tenant_id, kind, doc_type, current restricts	3 (writes)	4 (reads)
Spanner documind (GraphNode, GraphEdge with an embedding column)	the knowledge graph, seeded by meaning	4	-
Cloud SQL, the chat instance	LangGraph's PostgresSaver checkpoints; ADK's DatabaseSessionService	9	-
Cloud SQL, the gateway instance	LiteLLM spend logs and tag budgets	13	14
BigQuery documind_observability	the log sink and the tenant_daily view	14	8 (the admin console)
BigQuery rag_data (chunk_source, chunk_metadata, ingest_events, index_feed)	the chunk mirror, the feature job, the Dataplex scan	3 (writes)	14 (features), 7 (make-evalset)
RAG Engine corpus, Vertex AI Search data store	managed mirrors of current versions, for any tenants only	4	3 (the mirror step)

Store	What it holds	Owned by	Also read in
Buckets: uploads, datasets, media, tts-cache, audit (retention-locked)	the documents, the frozen trainsets and the GGUF, generated media, cached speech, the audit trail	2	3, 11, 12, 8
Chroma directory (the Rs 0 lane)	the same chunks and ids as Firestore, on the laptop	1	9, 10
Cloud Logging	the usage rows, thirty days	5 (writes)	14

5. What the plan borrows from other courses

Surveyed on 21 September 2026: DeepLearning.AI (Advanced RAG, LangGraph, MCP, A2A, Multimodal RAG, Finetuning, Efficiently Serving LLMs, Agentic AI), Kaggle's two 5-day intensives, Google's Production-Ready AI path and the InstaVibe / ADK / Cloud Run GPU codelabs, Hugging Face (Agents, MCP, smol), LangChain Academy (Intro to LangGraph, Ambient Agents), Anthropic Academy, Microsoft's two Beginners repos, Maven (AI Evals, Systematically Improving RAG, Mastering LLMs), FSDL. The field settles on 6-10 modules of 4-8 lessons; single-codebase courses keep modules self-contained with a fixed module template and a deployed, verified checkpoint at the end of every step.

Idea	Where it comes from	Where it lands here
A fixed module template	LangChain Ambient Agents (Intro / Resources / Core / Feedback), Microsoft Beginners	the four beats, section 1
"Why X" then "Architecture" before any code	DeepLearning.AI MCP and A2A courses (lessons 2 and 3 of each)	the Why and Where beats
Show the finished slice and its diagram first	DeepLearning.AI Multimodal RAG (lesson 2), InstaVibe step 2 with a service list per step	1.1, and every Where lesson
Evaluation early, then reused by every technique	Advanced RAG (the triad before the techniques), Anthropic Academy (section 2), Ng's Agentic AI (module 4 of 5), HF smol (unit 2)	Module 7 before the surfaces; every later Do lesson ends on the gate
From scratch, then rebuild in the framework	DeepLearning.AI LangGraph (lesson 2 to 3), HF Agents (unit 1 to 2)	9.3 direct loop to 9.4 LangGraph and ADK, same tool and same question
The same thing on three runtimes in one module	Google Production-Ready AI (Agent Engine / Cloud Run / GKE; vLLM vs Ollama)	4.5 (four backends), 9.5 (four brains), 13.5 (three engines)
An operate-it step: load it, watch it scale, read the bill	Cloud Run GPU Lab 3, DeepLearning.AI ADK "Productionize"	the Do beat's "price it", 13.5, 14.4, 15.4
A homework chain that breaks the app	Maven AI Evals (Recipe Bot: prompt, synthetic data, judge, retrieval eval, per-state diagnostics)	7.4 "make it go red", the chaos drills in 4.5 and 15.4
A checkpointed start state per lesson	DeepLearning.AI MCP and A2A (each notebook starts from the last one's finished code)	the kit at one state per module (a tag), never "continue from your own repo"
A separate "deployed" unit after the local one	HF MCP course (unit 2 local, unit 3 deployed)	1.4 on the laptop, then 10.4 on Cloud Run for MCP; 9.5's laptop switch
One primitive per lesson	ADK Crash Course	9.3 / 9.4, 10.2 / 10.3
A "when and why" gate before training	DeepLearning.AI Finetuning (lesson 2), Parlance Mastering LLMs (workshop 1)	12.1

6. Decisions the plan needs from you

- React.** The kit's UI is Streamlit (`services/frontend/`, twelve files); nothing in `code/` is React. A plan "based on the deploy code only" can teach Streamlit today (8.4) or wait for a React client to exist. If a React client is wanted, it is new code first - a thin client over `/v1/stream`, `/v1/sources` and `/v1/media/*`, behind the same IAP - and 8.4 becomes two lessons once it lands.

2. **VLM.** The kit uses a vision-language model in one way: Gemini describes an image or a video at ingest and the caption is what retrieval sees (11.1). It does not tune a vision model. If "VLM" means more than that, it is new code.
3. **The small model's training notebook.** `code/` holds the dataset (`evals/sft/`), the Modelfile generator and the Ollama image, but not the T4 notebook that produced the GGUF (it was v2's 10.5 and was deleted with it). 12.3 either re-authors it as this course's one Colab training lesson or teaches with the `SLM_STOCK=gemma3:4b` stand-in.
4. **The count.** 64 lessons at the beat times is about 90 hours. The two dials in section 2 give 60 or 56.
5. **Module 7's position.** Evaluation before the surfaces (this plan) or inside Module 12 as v3 had it.
6. **Cloud Shell versus Colab.** Modules 2, 3, 13, 14 and 15 are make-driven. The rest can run from Colab against the deployed lane, with the Rs 0 lane as the fallback for 9 and 10. Say if Colab-only lessons are a hard rule.

7. Before authoring

- `course-manifest.json` needs the v4 numbering, and `tools/check_colab_links.py` / `check_contract.py` need to know which lessons are make-driven (no notebook).
- The v3 folder skeletons under `Module 1` to `Module 8` (41 README files, one authored page: 3.3 Ingesting) map to v4 as follows: v3 1.x to Module 1 and 2.1-2.3 to Module 2; 3.1-3.2 to 1.1 and 3.1; 3.3 to Module 3 (the authored page splits across 3.2-3.4); 3.4 to Module 4; 3.5 to Module 5; 3.6 to Modules 6 and 14; 4.x to Module 10; 5.x to Module 11; 6.1-6.3 to Module 12 and 6.4-6.5 to Module 7; 7.1-7.4 to Module 9 and 7.5 to Module 10; 8.x to Module 13; 2.4-2.6 to Modules 8, 14 and 15.
- The v3 "still to do" list stands: the `deploy/` to `code/` rename in tools and docs, the extractor model, and a live rehearsal of `make up` before Modules 3-15 are written against its outputs.